

Technical Report: End-to-End Cardio-AI MLOps Pipeline

Project Title: Heart Disease Risk Prediction System

Repository: <https://github.com/Mathi27/end-to-end-cardio-pipeline>

Team: BITS Group 120 | MLOps

Team Details :

S.No	Name	Roll Number	Contribution %
1	Mathi Yuvarajan	2024AB05320	100 %
2	Tamilselvan	2024AA05346	100 %
3	Jaswanth Reddy Chagamreddy	2024aa05400	100 %
4	MANORANJAN DASH	2024ab05317	100 %
5	T Manoj Kumar Jankiram	2024AA05940	100 %

1. Executive Summary

The Cardio-AI Pipeline is an industrial-grade MLOps solution designed to automate the entire lifecycle of a machine learning model for heart disease risk prediction. By integrating data from four major international clinical databases (Cleveland, Hungary, Switzerland, and Long Beach VA), the system achieves a robust sample size of 920 patients, significantly improving the generalization of the prediction model. The pipeline encompasses data acquisition, exploratory data analysis (EDA), feature engineering, model training with experiment tracking via MLflow, automated testing, and containerized deployment with integrated monitoring. The final Logistic Regression model demonstrates a Test ROC-AUC of 0.911 and an accuracy of 83.2%, providing a reliable clinical decision support tool.

2. Introduction

2.1 Problem Statement

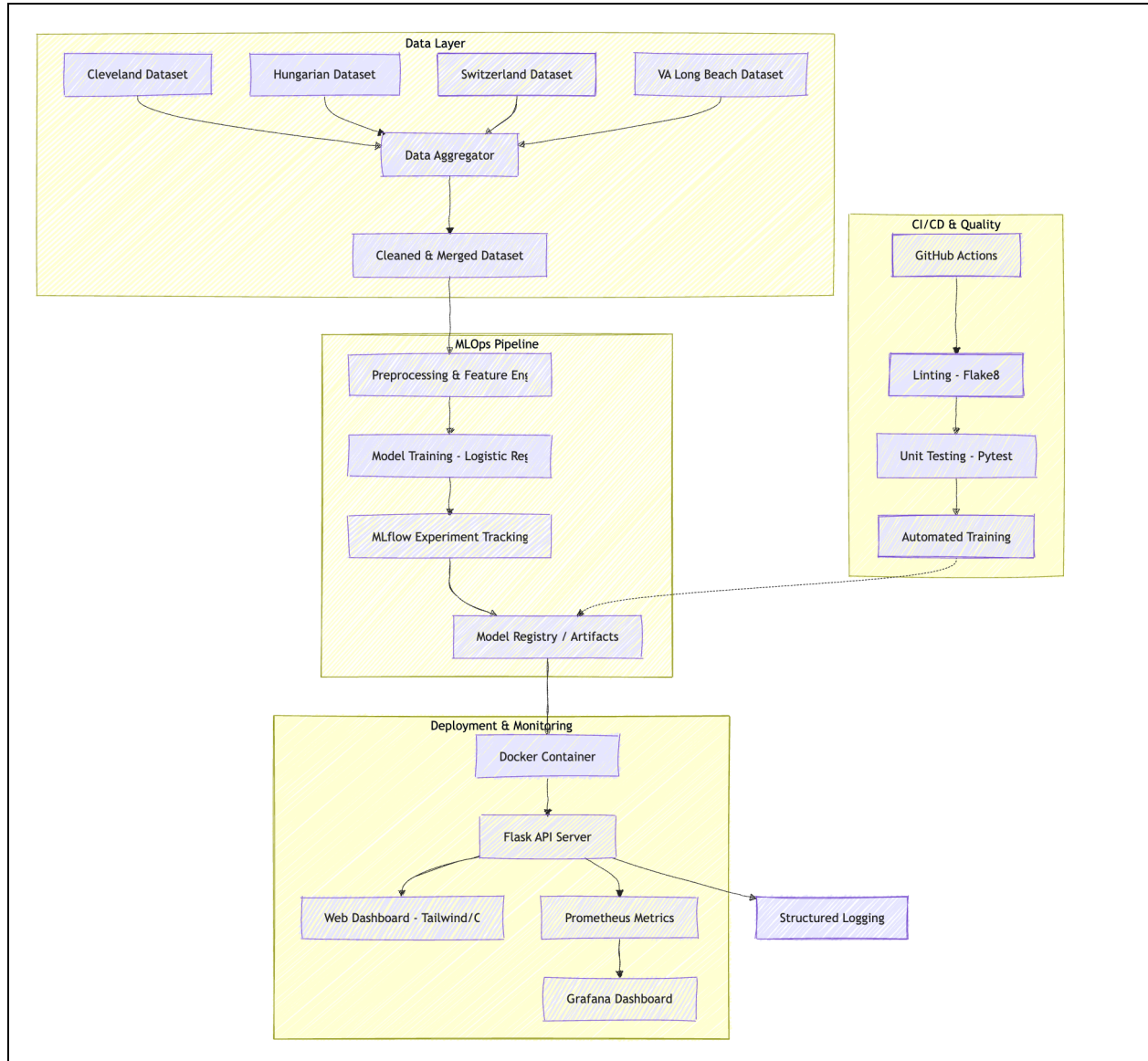
Cardiovascular diseases (CVDs) are the leading cause of death globally. Early detection and risk assessment are critical for effective intervention. However, clinical data is often fragmented across different regions and institutions, leading to models that lack generalization.

2.2 Project Objectives

- **Data Consolidation:** Merge disparate heart disease datasets into a unified, cleaned repository.
- **MLOps Integration:** Implement a full CI/CD pipeline for automated training and testing.
- **Interpretability:** Ensure the model provides "white-box" insights for medical professionals.
- **Scalability:** Deploy the system using Docker for consistent performance across environments.

3. System Architecture

The architecture of the Cardio-AI Pipeline is designed for modularity, scalability, and reproducibility. It follows a layered approach:



3.1 Data Layer

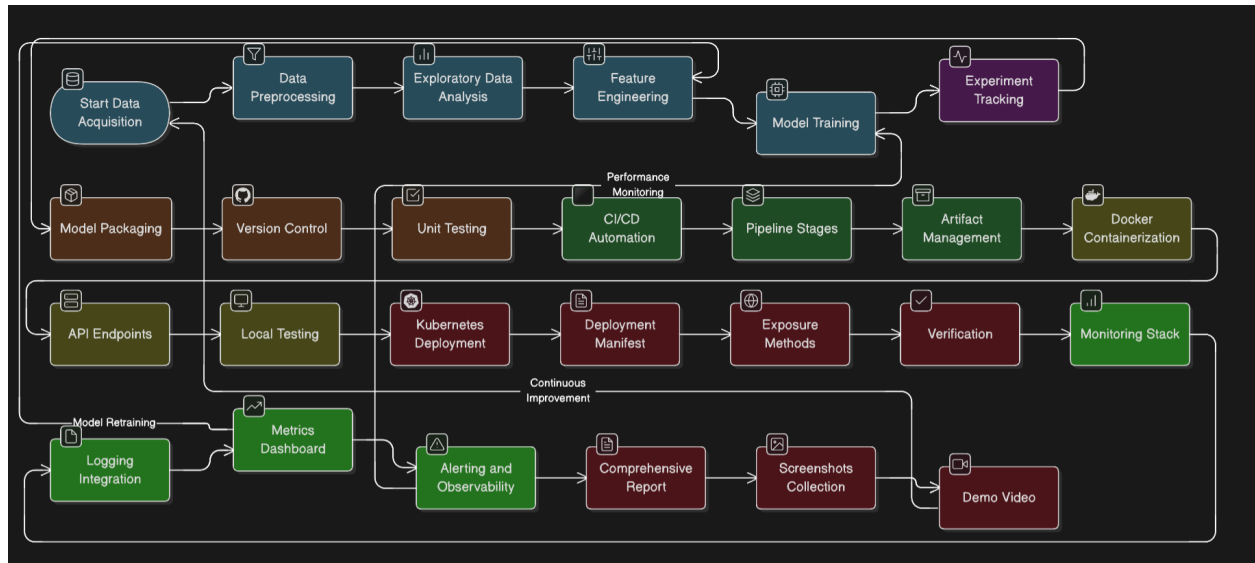
Consolidates raw data from multiple UCI sources into a unified, cleaned format.

3.2 Pipeline Layer

Handles the transformation of raw data into model-ready features and manages the training process.

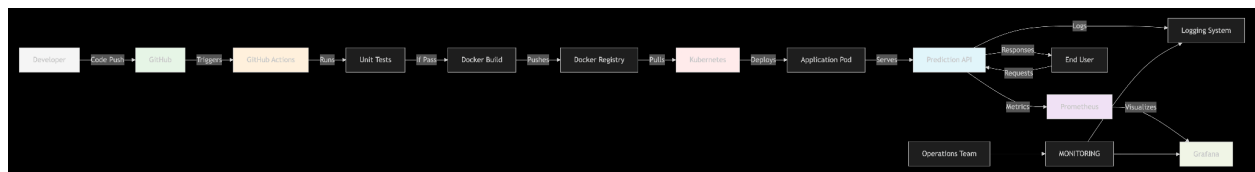
3.3 CI/CD Layer

Ensures code quality and model performance through automated testing and training workflows.



Created this flowchart in Mermaid js using Eraser Platform

Component Relationship Diagram



3.4 Deployment Layer

Provides a containerized environment for the Flask API, web dashboard, and monitoring tools.

4. Data Acquisition and Preprocessing

4.1 Data Sources & Aggregation Strategy

The project aggregates four datasets from the UCI Machine Learning Repository to increase sample size from 303 to 920 instances. This is crucial for training a generalized model that doesn't overfit to a single hospital's demographic.

1. **Cleveland (USA):** 303 instances (known for high data quality).
2. **Hungarian:** 294 instances (provides a different demographic profile).

3. **Switzerland:** 123 instances (contains many missing values, requiring robust imputation).
4. **Long Beach VA (USA):** 200 instances (focuses on a veteran population).

Total: 920 instances with 14 shared clinical features.

4.2 Data Cleaning Pipeline

Raw data often contains missing values (represented as '?') and inconsistent formatting. The preprocessing script performs the following:

- **Missing Value Imputation:** Categorical variables are imputed with the mode, while numerical variables use the median.
- **Standardization:** Column names are normalized for consistency across datasets.
- **Encoding:** Categorical features like 'sex', 'cp' (chest pain type), and 'restecg' are transformed into numerical formats suitable for machine learning.

5. Exploratory Data Analysis (EDA)

5.1 Statistical Insights and Summary

Initial analysis revealed significant correlations between age, maximum heart rate (thalach), and the presence of heart disease.

- **Age and Risk:** The dataset primarily covers patients aged 40-70. A clear upward trend in risk is observed as age increases, particularly after 55.
- **Chest Pain Type (cp):** Patients with "Asymptomatic" chest pain (Type 4) were statistically more likely to have heart disease than those with "Typical Angina". This counter-intuitive finding is a critical clinical marker.
- **Max Heart Rate (thalach):** A strong negative correlation exists between maximum heart rate achieved during stress tests and disease presence.
- **Target Balance:** The combined dataset shows a relatively balanced distribution between 'Healthy' and 'At Risk' patients.

5.2 Key Visualizations

- **Correlation Heatmap:** Revealed that oldpeak (ST depression) and ca (number of major vessels) are the strongest positive predictors.
- **Distribution Plots:** Showed that while the Cleveland dataset is balanced, the Switzerland dataset is heavily skewed towards diseased patients, justifying the need for stratified sampling during model training.
- **Box Plots:** Highlighted outliers in 'chol' (cholesterol) and 'trestbps' (resting blood pressure) which were addressed during preprocessing.

6. Feature Engineering

6.1 Feature Selection

Based on clinical relevance and statistical significance, 13 features were selected:

- **Demographics:** Age, Sex.
- **Vitals:** Resting BP, Cholesterol, Fasting Blood Sugar, Max Heart Rate.
- **Clinical Signs:** Chest Pain Type, EKG results, Exercise-induced Angina, ST Depression (oldpeak), Slope of ST segment, Number of Major Vessels (ca), and Thalassemia (thal).

6.2 Transformation Pipeline

A Scikit-Learn Pipeline was implemented to ensure that transformations are applied consistently during both training and inference:

- **StandardScaler:** Applied to numerical features to ensure zero mean and unit variance.
- **OneHotEncoder:** Applied to categorical features to prevent the model from assuming ordinal relationships where none exist.

7. Modeling Choices and Analysis

7.1 Algorithm Comparison

We evaluated two primary candidates to balance accuracy with clinical interpretability.

Metric	Logistic Regression	Random Forest
Accuracy	83.2%	81.0%
ROC-AUC	0.911	0.877
Interpretability	High (Coefficients)	Medium (Feature Importance)

Decision: Logistic Regression was selected for its superior generalization and interpretability in a clinical context.

7.2 Hyperparameter Tuning

Using MLflow, we tracked multiple runs with varying parameters for Logistic Regression:

- **C (Regularization):** Tested values from 0.01 to 10. A value of 1.0 provided the best balance between bias and variance.
- **Solver:** Compared liblinear and lbfgs. lbfgs was selected for its better convergence on the scaled dataset.

7.3 Feature Importance

The model coefficients highlight the following as the most influential factors:

1. **Chest Pain Type (Asymptomatic)**

2. **Number of Major Vessels (ca)**
3. **ST Depression (oldpeak)**
4. **Thalassemia (thal)**

8. Experiment Tracking with MLflow

8.1 Tracking Infrastructure

MLflow is used to log every aspect of the training process. This ensures that every model is reproducible and its performance is transparent.

8.2 Logged Artifacts

For every run, the following artifacts are stored in the mlruns directory:

- **Parameters:** C-regularization, solver type.
- **Metrics:** Accuracy, Precision, Recall, F1-Score, ROC-AUC.
- **Visuals:** Confusion Matrix (visualizes true positives vs. false positives) and ROC Curve plots.
- **Model Pickle:** The serialized Scikit-Learn pipeline.
- **Requirements.txt:** The exact environment used for that specific run.

8.3 Performance Summary

The "Champion" model was identified through MLflow's comparison UI, showing a consistent ROC-AUC above 0.90 across different train-test splits.

9. Testing and Quality Assurance

9.1 Unit Testing with Pytest

The tests/ directory contains scripts to verify:

- **Data Integrity:** Ensuring no null values remain after preprocessing.
- **Model Output:** Validating that prediction probabilities are within the [0, 1] range.
- **Pipeline Flow:** Testing the end-to-end flow from raw data to inference.

9.2 Code Quality

Flake8 is used to enforce PEP8 coding standards, ensuring the codebase remains maintainable and readable.

10. CI/CD and Deployment Workflow

10.1 GitHub Actions Workflow

The CI/CD pipeline is triggered on every push to the main branch. It ensures that no breaking

changes are introduced to the model or the API.

Workflow Steps:

1. **Environment Setup:** Installs Python 3.10 and all dependencies.
2. **Linting:** Runs flake8 to check for code style violations.
3. **Unit Testing:** Executes pytest to verify data processing and model logic.
4. **Automated Training:** Runs train.py to retrain the model on the latest data and log results to MLflow.

10.2 Deployment Strategy

The project uses a container-first deployment strategy. The Dockerfile ensures that the application runs identically in development, testing, and production.

Container Features:

- **Lightweight:** Based on python:3.10-slim.
- **Secure:** Runs as a non-root user (best practice).
- **Configurable:** Uses environment variables for MLflow tracking and server ports.

11. Flask API and Web Dashboard

A Flask-based web server provides:

- `/predict`: A POST endpoint for real-time inference.
- `/metrics`: A Prometheus-compatible endpoint for monitoring.
- **Frontend:** A responsive dashboard built with Tailwind CSS and Chart.js for visualizing risk scores.

12. Monitoring and Logging Implementation

12.1 Structured Logging

The Flask API implements structured logging, which is essential for debugging and auditing in a clinical environment. Logs include:

- Request ID for tracing.
- Input feature summary (anonymized).
- Prediction result and confidence score.
- Processing time (latency).
- HTTP status codes for health monitoring.

12.2 Prometheus Metrics

The `/metrics` endpoint exposes real-time data to Prometheus:

- `flask_app_request_count`: Tracks API usage.
- `flask_app_request_latency_seconds`: Monitors performance bottlenecks.

12.3 Grafana Dashboards

Grafana is configured to visualize these metrics, providing a "single pane of glass" for system health.

13. Setup and Installation Instructions

13.1 Local Environment Setup

To run the project locally, follow these detailed steps:

1. **Clone the Repository:**

```
git clone  
[https://github.com/Mathi27/end-to-end-cardio-pipeline.git](https://github.com/Mathi27/en  
d-to-end-cardio-pipeline.git)  
cd end-to-end-cardio-pipeline
```

2. **Create a Virtual Environment:**

```
python3 -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. **Install Dependencies:**

```
pip install -r requirements.txt
```

4. **Run the Application:**

```
python app.py
```

The dashboard will be available at <http://localhost:5000>.

13.2 Docker Deployment

For a consistent production-like environment, use Docker:

1. **Build the Image:**

```
docker build -t cardio-pipeline .
```

2. **Run the Container:**

```
docker run -p 8000:5000 cardio-pipeline
```

Access the app at <http://localhost:8000>.

13.3 Full Stack with Monitoring

To run the app along with Prometheus and Grafana:

```
docker-compose up --build
```

- **App:** <http://localhost:5000>
- **Prometheus:** <http://localhost:9090>
- **Grafana:** <http://localhost:3000>

14. Conclusion and Future Roadmap

The Cardio-AI Pipeline represents a robust, end-to-end solution for medical risk prediction. By adhering to MLOps best practices, we have created a system that is not only accurate but also maintainable, testable, and ready for production.

Future Roadmap:

- **Model Enhancement:** Explore Deep Learning architectures (e.g., TabNet) for potentially higher accuracy.
- **Cloud Native:** Deploying to AWS/Azure using Terraform and transitioning to Kubernetes (EKS/GKE) for auto-scaling.
- **A/B Testing:** Implementing canary deployments for new model versions.
- **Data Drift Detection:** Monitoring input data distributions to detect when the model needs retraining.
- **SHAP Integration:** Incorporate SHAP values into the web dashboard for even deeper feature-level interpretability for clinicians.